

Lab 4 – Capstone: Orchestrating an Airflow & Spark Data Pipeline

Big Data Processing – EPF Engineering School

Théo Burdinat `theo.burdinat1@epf.edu.fr`

June 5 & 10, 2026 | Pair work | English

Kit: `lab4_student.zip` on Moodle. Run `Setup_TP4.md` before the lab.

Context and goal

The situation

A retail partner drops **one CSV file per day** of store transactions. Operations wants a **daily KPI dashboard**: revenue and transaction counts by category and country. In Lab 3 you already built the **orchestration shell**: wait for the file, ingest to Parquet, publish a report path. Lab 4 adds the **analytics layer**: PySpark computes the KPIs; Airflow still decides *when* each step runs and what happens on failure.

What we want at the end

For a logical day `ds` (e.g. 2026-06-03), a successful capstone run should:

1. Find `data/incoming/transactions_<ds>.csv` (dropped by the vendor simulator).
2. Produce typed silver Parquet under `data/raw/dt=<ds>/`.
3. Run **your** Spark job: curated KPI Parquet under `data/curated/dt=<ds>/` and a small JSON report `data/reports/dashboard_<ds>.json` (totals + paths for the UI).
4. Expose this through **your** Airflow DAG (≥ 5 tasks), with a visible failure when data is bad.
5. Support **idempotence**: triggering the same `ds` again must not duplicate KPI rows or JSON metrics.

You **design** the graph and implement `include/team_<shortname>_spark.py`.

Simulating the vendor: `scripts/vendor_drop.py`

`vendor_drop.py` plays the upstream vendor: it writes CSVs into `data/incoming/` on your laptop. **Run it on the host** (outside Docker), from the extracted `lab4_student/` folder:

```
python scripts/vendor_drop.py --seed-pack --volume small
python scripts/vendor_drop.py --reference
```

- **Normal use:** creates `transactions_YYYY-MM-DD.csv` with realistic columns (`tx_id`, `category`, `country`, `amount_eur`, ...).
- **Deterministic:** same date $\Rightarrow$ same data (handy for debugging and demos).
- **Then:** your Airflow `FileSensor` waits for that file; the DAG ingests and runs Spark for `ds`.

Option	When you use it in the lab
<code>-date 2026-06-03</code>	Drop a single day before testing one ds in the UI
<code>-seed-pack</code>	Create June 1–14, 2026 in one go (typical first setup)
<code>-volume small / medium</code>	Row count (default small $\approx$ 200 rows; enough on a laptop)
<code>-corrupt</code>	All amount_eur = 0 — use to test validate and a red task in Airflow
<code>-reference</code>	Writes data/reference/category_targets.csv (optional join target for Track S)
<code>-delay 60</code>	Waits before writing — simulates a late vendor file (sensor / SLA thinking)
<code>-range START:END</code>	Several days without calling <code>-seed-pack</code>

If the sensor never turns green, the usual mistake is: **no CSV for that ds** — run **vendor_drop** for that date first.

Target architecture (what each layer should contain)

Layer	Path (one day ds)	Content	Who builds it
Bronze	data/incoming/	Raw CSV “as delivered”	vendor_drop.py
Silver	data/raw/dt=<ds>/	Clean Parquet, one partition per day	Provided ingest_day (DuckDB)
Gold	data/curated/dt=<ds>/	KPI aggregates (e.g. by category & country)	Your PySpark module
Serve	data/reports/	dashboard_<ds>.json for a mock BI screen	Your pipeline (written after Spark)

Deliverables

Provided (examples)	You create (graded)	Minimum
DAG lab4_starter	dags/team_*.py	$\geq$ 5 Airflow tasks (you create)
Smoke-test Spark (Docker image)	include/team_<name>_spark.py	$\geq$ 3 Spark transforms (you create)
Templates docs	Implement & extend	Templates unchanged = fail
Tracks R, S, O, Q, P, X	Optional	Not required to pass

Airflow — at least 5 tasks

The reference DAG **lab4_starter** (file **dags/starter_pipeline.py**) has only **4 tasks**.

Your capstone DAG (**dags/team_*.py**) must define **at least 5 new Airflow tasks** (sensor, **@task**, or operator — each counts as one task). Each task must have a **clear role** documented in **README_TEAM.md** (wait, ingest, validate, compute, publish, alert, ...).

Examples of tasks (choose your own names and order):

- **validate** — fail if silver data is corrupt
- **notify_on_failure** — marker file or log on failure
- **export_summary** — copy JSON to a second location

Spark (≥ 3 transformations you implement)

Copy `include/team_spark_TEMPLATE.py` to `include/team_<name>_spark.py`:

- **Three functions** (e.g. `transform_1`, `transform_2`, `transform_3`): `read` $\rightarrow$ `enrich` $\rightarrow$ `aggregate`.
- `run_daily(logical_date)` called from your Airflow Spark task.
- Outputs: Parquet under `data/curated/dt=<date>/` and `data/reports/dashboard_<date>.json`.

Design your three transforms from the spec above (`read` $\rightarrow$ `enrich` $\rightarrow$ `aggregate`); the starter smoke test does not ship reference Spark source in the kit.

Also required

- Idempotent re-run for one `execution_date` (ds)
- Failure visible in the Airflow UI (demo with `vendor_drop -corrupt` recommended)
- `README_TEAM.md` in English: all tasks, all Spark steps, diagram, backfill command

Schedule

When	Time	What
June 5	CM4 + Part A (3 h)	Smoke test; plan; start your DAG and Spark module
June 5–9	~3 h	Part B: finish capstone; optional tracks R–X
June 9	23:59	Push Git + upload slides on Moodle (repo URL on slides)
June 10	15 min + Q&A	Part C: demo + defense (English)

Grading

Criterion	%
Technical (pipeline, your Spark, orchestration, README)	50%
Defense (explain what & why ; live demo ~5 min)	50%

Submission (deadline: June 9, 23:59)

Deadline: Monday June 9, 2026, 23:59 (before Part C on June 10).

Git — one repository per pair

Each pair creates its own repository (GitHub). The instructor does **not** provide a shared course repo.

- **Public repository** — the instructor can clone/read without an invite, **or**
- **Private repository** — add `theoburdinat` on GitHub with **read** access.
- Suggested tree at repo root: `days/`, `include/`, `README_TEAM.md` (same layout as inside the ZIP, but **without** committing the whole Moodle archive).
- **Required paths:** `days/team_<shortname>.py`, `include/team_<shortname>_spark.py`, `README_TEAM.md`, optional `demo_backup/` for screenshots.
- Default branch `main` (or `master`); push before the deadline.

Moodle — submit (deadline June 9, 23:59)

- **One upload per pair:** Part C slides (PDF or PPTX) on Moodle
- Slides must include: context, architecture, task table (≥ 5), Spark steps (≥ 3), demo plan
- **On the slides:** your pair's Git repo **HTTPS clone URL** (title slide or architecture slide)

Part A – June 5 in class (3 hours)

Important

Complete `Setup_TP4.md` before the lab. Run `lab4_starter` only to verify Docker, Java, and PySpark.

A0 – Smoke test

Run `python scripts/smoke_test.py` (or trigger DAG `lab4_starter` for 2026-06-01 in the UI).

Done when: `data/reports/dashboard_2026-06-01.json` exists with `"status": "smoke_ok"` (PySpark config check only).

A1 – Plan

A1.1 Copy `team_TEMPLATE.py` → `dags/team_<name>.py`; `team_spark_TEMPLATE.py` → `include/team_<name>_spark.py`

A1.2 List your Airflow tasks (≥ 5 ; names + role in the pipeline).

A1.3 List ≥ 3 Spark transformations (with a short description).

A1.4 Start `README_TEAM.md` (template in kit).

A2 – Start the capstone

Goal: a working skeleton, not necessarily a finished capstone.

- ☐ ≥ 5 **Airflow tasks** wired in `dags/team_<name>.py` (validate uncommented is enough for 5)
- ☐ ≥ 3 **Spark transforms** started in `include/team_<name>_spark.py`
- ☐ At least one green run for one `ds` (two dates + idempotence check → Part B)

A3 – Tracks

Pick optional tracks for Part B ; describe you additional work in `README_TEAM.md`. The more tracks you complete, the better your grade will be (you will get questions about different concepts, don't use ChatGPT to make all the work for you).

Part B – Autonomy (June 5–9, ~3 hours)

Finish the mandatory checklist in Part A, push to Git and upload slides on Moodle (deadline **June 9, 23:59**), then optional tracks. Rehearse Part C.

```
docker compose exec airflow-scheduler \
  airflow dags backfill team_yourname -s 2026-06-01 -e 2026-06-07 --reset-dagruns
```

Exploration tracks (optional)

ID	Name	Examples
R	Reliability	Retries, callbacks, sensor timeout, backfill docs
S	Spark depth	Strict schema, broadcast join, shuffle notes
O	Orchestration	TaskGroups, dynamic mapping, Datasets
Q	Data quality	Row-count bounds; demo <code>-corrupt</code>
P	Custom	Your proposal + README design paragraph
X	SparkSubmit	Cluster-style submit (Appendix B)

Part 1: Part C – Defense (June 10)

Slot: 15 min presentation + 5–10 min Q&A per pair (**English**).

Present your Airflow tasks (≥ 5) and Spark steps (≥ 3) **before** the demo. The jury grades **understanding**: for each major choice, be ready to say **what you did** and **why** (not a line-by-line code walkthrough).

Q&A: Each student answers **several questions** individually (typically 2–4), covering orchestration, Spark, idempotence, and failure modes — not a single question per person.

Backup if live demo fails

Prepare a small backup folder (or slides): Airflow graph screenshot (green run), `dashboard_<ds>.json` or curated path, optional red task after `-corrupt`. If Docker fails on June 10, you can still defend with screenshots — but **try the live demo first**.

1. **Context** — business problem (~2 min)
2. **Architecture** — diagram; table of tasks and Spark steps (~3 min)
3. **Live demo** — one happy path; optional `-corrupt` failure (~5 min)
4. **Technical choices** — idempotence, validate-before-Spark, one Spark decision (~3 min)
5. **Production & tracks** — `local[*]` vs cluster; tracks attempted (~2 min)

Example questions: How many Airflow tasks in your DAG? Re-run same `ds`? Why validate before Spark? What does `vendor_drop -corrupt` change?

Part A: README_TEAM.md template

```
# Team / DAG id: team_<shortname>

## Airflow (5 tasks)
| task_id | Role |
|-----|-----|
| ... | ... |

## Spark (>=3, include/team_<shortname>_spark.py)
1. transform_1: ...
2. transform_2: ...
3. transform_3: ...
```

```
## Idempotence / backfill / failure demo
## Tracks attempted (optional)
## Demo script (June 10, ~5 min live)
## Demo backup (screenshots if live demo fails)
```

Part B: Track X – SparkSubmitOperator

- **Capstone Spark code** lives in `include/team_<name>_spark.py` (called from a `@task`).
- `spark_jobs/daily_kpis.py` is a **CLI wrapper** for `spark-submit` only (Track X).
- In production, `SparkSubmitOperator` submits `spark-submit` to a cluster; Airflow workers only monitor status.
- Document in `README_TEAM.md` why you kept or dropped in-process `SparkSession` on a laptop.